technologies

MDPI

Supplementary Materials

A. Benchmark Data Models

This appendix defines the benchmark metamodels used by the MVCL controlled automotive benchmark scenario. The definitions are aligned with the executable rule set in Appendix B so that every rule refers only to declared classes, attributes, references, and cardinalities.

A.1 Electrical Metamodel (Electrical.ecore)

```
@namespace(uri="http://www.mvcl.org/electrical", prefix="electrical")
package electrical;
class ElectricalComponent {
attribute id: EString;
attribute name: EString;
attribute type: ComponentType;
attribute voltage: EDouble;
attribute current: EDouble;
attribute powerRating: EDouble;
reference connectedTo: ElectricalComponent[0..*];
}
enum ComponentType {
BATTERY;
RESISTOR;
CAPACITOR;
MOTOR;
SENSOR;
CONTROLLER;
POWER_SUPPLY;
}
class Battery extends ElectricalComponent {
attribute capacity: EDouble;
attribute chemistry: BatteryChemistry;
attribute cycles: EInt;
attribute health: EDouble;
reference supplies: PowerSupply[0..1];
}
enum BatteryChemistry {
LITHIUM_ION;
LEAD_ACID;
NICKEL_METAL_HYDRIDE;
SOLID_STATE;
}
class PowerSupply extends ElectricalComponent {
attribute maxPower: EDouble;
attribute efficiency: EDouble;
attribute inputVoltage: EDouble;
attribute outputVoltage: EDouble;
reference powers: SystemLoad[0..*];
```

```
}
class SystemLoad extends ElectricalComponent {
attribute peakLoad: EDouble;
attribute averageLoad: EDouble;
attribute priority: LoadPriority;
reference connectedTo: PowerSupply[1..*];
}
enum LoadPriority {
CRITICAL;
HIGH;
MEDIUM;
LOW;
BACKUP;
}
```

A.2 Software Metamodel (Software.ecore)

```
@namespace(uri="http://www.mvcl.org/software", prefix="software")
package software;
class SoftwareComponent {
attribute id: EString;
attribute name: EString;
attribute version: EString;
attribute vendor: EString;
attribute deploymentDate: EDate;
reference requires: Requirement[0..*];
reference implements: Functionality[0..*];
}
class Requirement {
attribute id: EString;
attribute description: EString;
attribute type: RequirementType;
attribute priority: RequirementPriority;
attribute minRequiredCapacity: EDouble;
attribute minResponseTime: EDouble;
attribute maxLatency: EDouble;
attribute isMandatory: EBoolean;
reference assignedTo: SoftwareComponent[1];
}
enum RequirementType {
FUNCTIONAL;
NON_FUNCTIONAL;
PERFORMANCE;
SECURITY;
RELIABILITY;
USABILITY;
}
enum RequirementPriority {
MUST_HAVE;
SHOULD_HAVE;
COULD_HAVE;
WON_T_HAVE;
```

```
}
class Functionality {
attribute id: EString;
attribute name: EString;
attribute description: EString;
attribute complexity: ComplexityLevel;
reference providedBy: SoftwareComponent[1];
reference dependsOn: Functionality[0..*];
}
enum ComplexityLevel {
LOW;
MEDIUM;
HIGH;
VERY_HIGH;
}
class Controller extends SoftwareComponent {
attribute firmwareVersion: EString;
attribute updateStatus: UpdateStatus;
attribute lastValidationDate: EDate;
attribute controlLoopFrequency: EDouble;
reference linkedSensor: Sensor[0..*];
reference controls: Actuator[0..*];
}
class Actuator {
attribute id: EString;
attribute name: EString;
}
class Sensor {
attribute id: EString;
attribute type: SensorType;
attribute accuracy: EDouble;
attribute range: EDouble;
attribute units: EString;
reference connectedTo: Controller[1];
}
enum SensorType {
TEMPERATURE;
PRESSURE;
CURRENT;
VOLTAGE;
SPEED;
POSITION;
}
enum UpdateStatus {
UP_TO_DATE;
UPDATE_AVAILABLE;
UPDATE_IN_PROGRESS;
UPDATE_FAILED;
}
```

A.3 Production Metamodel (Production.ecore)

```
@namespace(uri="http://www.mvcl.org/production", prefix="production")
package production;
class ProductionUnit {
attribute id: EString;
attribute name: EString;
attribute status: ProductionStatus;
attribute location: EString;
attribute capacity: EDouble;
attribute validated: EBoolean;
attribute startDate: EDate;
reference parts: MechanicalPart[0..*];
reference line: ProductionLine[1];
}
enum ProductionStatus {
DESIGN;
PROTOTYPE;
TESTING;
PRODUCTION;
MAINTENANCE;
DECOMMISSIONED;
}
class ProductionLine {
attribute id: EString;
attribute name: EString;
attribute type: LineType;
attribute speed: EDouble;
attribute efficiency: EDouble;
attribute validated: EBoolean;
attribute location: EString;
attribute lastValidationDate: EDate;
reference units: ProductionUnit[0..*];
reference parts: MechanicalPart[0..*];
}
enum LineType {
ASSEMBLY;
PAINTING;
TESTING;
PACKAGING;
QUALITY_CONTROL;
}
class MechanicalPart {
attribute id: EString;
attribute name: EString;
attribute partId: EString;
attribute type: PartType;
attribute material: EString;
attribute weight: EDouble;
attribute dimensions: EString;
attribute validated: EBoolean;
attribute inspectionDate: EDate;
reference usedIn: ProductionLine[1];
```

```
    reference linkedController: Controller[0..1];
    }
    enum PartType {
    STRUCTURAL;
    ELECTRICAL;
    MECHANICAL;
    ELECTRONIC;
    PLASTIC;
    COMPOSITE;
    }
```

B. MVCL Rule Set

The rule set below resolves the type, cardinality, and navigation inconsistencies identified during the audit of the previous draft. The intended distinction between syntactic, structural, and semantic inconsistency detection is preserved.

B.1 Syntactic Rules (syntactic_rules.mvcl)

```
                    // SR-01: Required attributes
rule SR_01_AttributeExistence : syntactic
context ElectricalComponent ec
check ec.id <> null and ec.name <> null
                    // SR-02: Non-negative requirement capacity
rule SR_02_TypeCompatibility : syntactic
context Requirement r
check r.minRequiredCapacity >= 0.0
                    // SR-03: Enumeration validity
rule SR_03_EnumValidity : syntactic
context ProductionUnit pu
check pu.status <> null
                    // SR-04: Reference validity
rule SR_04_ReferenceValidity : syntactic
context MechanicalPart mp
check mp.linkedController <> null implies mp.linkedController.id <> null
                    // SR-05: Battery attribute-domain consistency
rule SR_05_DataTypeConsistency : syntactic
context Battery b
check b.capacity >= 0.0 and b.chemistry <> null
                    // SR-06: Software component completeness
rule SR_06_SoftwareCompleteness : syntactic
context SoftwareComponent sc
check sc.vendor <> null and sc.version matches "[0-9]+\.[0-9]+\.[0-9]+"
                    // SR-07: Production unit completion
rule SR_07_ProductionCompleteness : syntactic
context ProductionUnit pu
check pu.startDate <> null and pu.capacity > 0
                    // SR-08: Controller firmware validity
rule SR_08_FirmwareValidity : syntactic
context Controller c
check c.firmwareVersion matches "[0-9]+\.[0-9]+"
```

B.2 Structural Rules (structural_rules.mvcl)

```
                    // ST-01: Required sensor connections
rule ST_01_RequiredConnection : structural
context Controller c
check c.linkedSensor->size() > 0
                    // ST-02: Production-line completeness
rule ST_02_LineCompleteness : structural
context ProductionLine pl
check pl.units->size() = pl.parts->size()
                    // ST-03: Controller mapping uniqueness
rule ST_03_ReferenceUniqueness : structural
context MechanicalPart mp
check MechanicalPart.allInstances()->isUnique(p | p.linkedController)
```

```
                        // ST-04: Bidirectional part-line consistency
    rule ST_04_BidirectionalConsistency : structural
    context ProductionLine pl
    check pl.parts->forAll(p | p.usedIn = pl)
                        // ST-05: Hardware-controller mapping
    rule ST_05_HardwareSoftwareMapping : structural
    context ElectricalComponent ec
    check ec.connectedTo->forAll(c |
                        c.type = ComponentType::CONTROLLER implies c.id <> null)
                        // ST-06: Controller-sensor linkage
    rule ST_06_ControllerSensorLink : structural
    context Controller c
    check c.linkedSensor->forAll(s | s.connectedTo = c)
                        // ST-07: Requirement assignment
    rule ST_07_RequirementAssignment : structural
    context Requirement r
    check r.assignedTo <> null
                        // ST-08: Dependency graph acyclicity
    rule ST_08_DependencyAcyclicity : structural
    context Functionality f
    check f.dependsOn->closure(d | d.dependsOn)->excludes(f)
                        // ST-09: Production unit-line relationship
    rule ST_09_ProductionUnitLine : structural
    context ProductionUnit pu
    check pu.line <> null and pu.line.units->includes(pu)
                        // ST-10: Part-line relationship
    rule ST_10_PartLineRelationship : structural
    context MechanicalPart mp
    check mp.usedIn <> null and mp.usedIn.parts->includes(mp)
```

B.3 Semantic Rules (semantic_rules.mvcl)

```
                        // SE-01: Battery capacity adequacy
    rule SE_01_BatteryCapacity : semantic
    context Battery b, Requirement r
    check b.capacity >= r.minRequiredCapacity
                        // SE-02: Power-supply adequacy with safety factor
    rule SE_02_PowerSupplyAdequacy : semantic
    context PowerSupply ps, SystemLoad sl
    check ps.maxPower >= sl.peakLoad * 1.2
                        // SE-03: Production readiness
    rule SE_03_ProductionReadiness : semantic
    context ProductionUnit pu
    check (pu.status = ProductionStatus::PRODUCTION)
                        implies (pu.validated = true and pu.capacity > 0)
                        // SE-04: Effective software-hardware capacity alignment
    rule SE_04_CapacityAlignment : semantic
    context Requirement r, Battery b
    check r.minRequiredCapacity <= b.capacity * b.health / 100
                        // SE-05: Validation timing constraint
    rule SE_05_ValidationTiming : semantic
    context Controller c
```

```
    check daysBetween(c.deploymentDate, c.lastValidationDate) <= 30
                        // SE-05 uses an implementation-defined daysBetween helper that returns elapsed
                    calendar days; the helper is provided by the MVCL-to-EVL mapping layer to avoid
                    relying on implicit date-minus-integer coercion.
                    // SE-06: Production-line efficiency
    rule SE_06_LineEfficiency : semantic
    context ProductionLine pl
    check pl.efficiency >= 75.0
                        // SE-07: Performance requirement versus sensor accuracy
    rule SE_07_SensorAccuracy : semantic
    context Sensor s, Requirement r
    check (r.type = RequirementType::PERFORMANCE and
                        r.assignedTo = s.connectedTo)
                        implies s.accuracy >= 95.0
                        // SE-08: Battery lifecycle constraint
    rule SE_08_BatteryLifecycle : semantic
    context Battery b
    check b.cycles < 1000
                        // SE-09: MUST_HAVE requirement has a versioned assigned component
    rule SE_09_RequirementPriority : semantic
    context Requirement r
    check (r.priority = RequirementPriority::MUST_HAVE)
                        implies (r.assignedTo.version <> null)
                        // SE-10: Production-unit location validity
    rule SE_10_LocationValidity : semantic
    context ProductionUnit pu
    check pu.location = pu.line.location
                        // SE-11: Controller sensor coverage
    rule SE_11_SensorCoverage : semantic
    context Controller c, SystemLoad sl
    check c.linkedSensor->size() >=
                        (sl.priority = LoadPriority::CRITICAL ? 2 : 1)
                        // SE-12: Power-supply redundancy
    rule SE_12_PowerRedundancy : semantic
    context SystemLoad sl
    check sl.connectedTo->size() >=
                        (sl.priority = LoadPriority::CRITICAL ? 2 : 1)
```

B.4 Tolerance Context Definitions (contexts.json)

```
    {
    "contexts": [
    {
    "id": "prototype-phase",
    "description": "Development prototype phase with relaxed constraints",
    "parameters": {
    "phase": "Prototype",
    "criticality": "Low",
    "deadline": "2026-12-31",
    "team_size": 5,
    "budget": 1000000
    }
```

```
    },
    {
    "id": "production-phase",
    "description": "Production phase with strict constraints",
    "parameters": {
    "phase": "Production",
    "criticality": "High",
    "deadline": "2026-09-01",
    "team_size": 20,
    "budget": 10000000
    }
    },
    {
    "id": "maintenance-phase",
    "description": "Maintenance phase with balanced constraints",
    "parameters": {
    "phase": "Maintenance",
    "criticality": "Medium",
    "deadline": "2026-12-01",
    "team_size": 8,
    "budget": 3000000
    }
    }
                        ]
}
```

The numerical tolerance thresholds for SE-01–SE-12 remain in the machine-readable contexts.json artifact used by the tolerance module. They are intentionally separated from the detection rules, preserving the detection–tolerance separation adopted by the framework.

C. Reproducible Benchmark Generation and Evaluation Protocol

C.1. Evaluation Objectives

The benchmark suite evaluates rule coverage, mutation effectiveness, and detection quality. Rule coverage is assessed by exercising all 30 inconsistency rules in Appendix B; mutation effectiveness is assessed by verifying that each mutation operator introduces its targeted inconsistency; and detection quality is assessed by comparing evaluator output with an independently generated ground-truth set.

C.2. Benchmark Model and Mutation Design

A clean multi-view automotive model instance is generated first and then subjected to a rule-specific mutation operator. Each of the 30 operators corresponds to one MVCL rule and records the benchmark identifier, rule identifier, affected element identifiers, mutation type, and expected violation.

C.3. Independent Ground Truth

Ground truth is generated independently from evaluator output. A true positive is a detected violation matching a ground-truth rule and affected element identifiers; a false positive is a reported violation without a ground-truth match; and a false negative is a ground-truth violation not reported by the evaluator. Ground-truth records are serialized as JSON and validated against the accompanying schema. For the tolerance experiment, the benchmark generator additionally produces reference tolerance labels through a separate deterministic policy function based on the predefined context-action mappings; these labels are independent of the GPT-5.6 responses and are not produced by the LLM. No human expert panel is used for the current tolerance labels.

C.4. Benchmark Configurations

Five benchmark configurations are used: B-100, B-500, B-1000, B-TOL, and B-COMP. The first three increase model scale; B-TOL emphasizes tolerance-related context; and B-COMP provides a larger composite stress case.

Benchmark	Realized Model Elements	Mutation Cases	Mutation-Construction Failures
B-100	139	30	0
B-500	696	30	0
B-1000	1,589	30	0
B-TOL	698	30	0
B-COMP	7,959	30	0

Five configurations are used: B-100, B-500, B-1000, B-TOL, and B-COMP. The first three increase model scale; B-TOL emphasizes tolerance-related context; and B-COMP provides a larger composite stress case.

Each benchmark exercises all 30 mutation operators. Zero failed mutation constructions establish generator-level coverage and are not interpreted as final MVCL detection accuracy.

C.5. Evaluation Metrics

For ground-truth set G and detection set D, TP denotes a detected violation matching a ground-truth rule and affected element; FP denotes a reported violation without a ground-truth match; and FN denotes a ground-truth violation not reported by the evaluator.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Runtime is recorded independently of correctness metrics and is reported only for executions of the actual MVCL implementation used in the paper.

C.6. Execution Protocol

The final pipeline is: (i) generate a clean model; (ii) apply one controlled mutation; (iii) serialize the mutated model; (iv) execute the actual MVCL/Xtext/EVL implementation; (v) serialize detected inconsistencies; and (vi) compare them with independent ground truth. The Python reference evaluator is used only for development and benchmark sanity checking.

C.7. Generator Sanity Check

The benchmark package successfully constructs 30 mutation cases for each of the five configurations with zero mutation-construction failures. Because randomly generated larger models may contain additional pre-existing violations, reference-evaluator precision values are not reported as final MVCL detection results. Final precision, recall, F1, and runtime values must be obtained from the actual implementation.

C.8. Reproducibility Artifacts

The artifacts are organized in the public GitHub repository https://github.com/yaser-shahvari-lgtm/MVCL-Benchmark. The repository contains the benchmark code and artifacts used for the publication; an archived release DOI can be added when available.

C.9. Threats to Validity and Scope

The benchmark models are synthetic and do not represent the full variability of industrial automotive systems. Mutation-based evaluation measures detection of predefined inconsistency patterns and does not establish completeness for all possible modeling errors. Future work should complement the benchmark with industrial models and naturally occurring inconsistencies.

C.10. Relation to the Main Evaluation

Appendix C provides the experimental infrastructure underlying Section 3. The main evaluation should report only measurements obtained from the actual MVCL implementation and should reference the benchmark identifiers defined here. This separation keeps the reference data independent of the system under test.